ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time

complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

- The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each
- How the same problem would be written in a procedural style, and what changes in an object-oriented style
- The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system
- The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)

You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks

Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one place only.

Discuss:

- The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples
- #define, const and constexpr constants, and why constexpr is preferred for the fare table
- enum and enum class for zones, and why enum class is safer
- Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

- The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system

- Why money should be stored in integer paise and not as double rupees
- Type compatibility rules, and what the compiler does with int, double and bool values in the same expression
- The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

- Integer division and integer-to-double conversion
- Comparison of signed and unsigned values, and the compiler warning it produces
- Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them
- static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
   int a = 7, b = 2;
   double c = a / b;
```

```
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';

    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';

    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (write before running):

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

- Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment
- The difference between x++ and ++x, and why i = i++ is a mistake
- The result of / and % with negative operands
- Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';

    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

My predicted output (write before running):

For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's journeys.

Discuss:

- When to use if–else, a switch on the zone, and the conditional operator
- How a for loop, a range-based for loop and a while loop could process the journeys, and which is best
- Where break and continue are needed (for example, skipping an invalid journey)
- What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream>


enum class Zone { Inner, Middle, Outer };
```

```
constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;     // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
   // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add
   //   PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only

}

bool canTravel(long long balance, long long f)
{
   // TODO 2: Return true only if the balance left after paying the fare is at least
   //   MIN_BALANCE_PAISE

}

int main()
{
   long long balance = 10000;                    // Rs 100.00 in paise
   struct Trip { Zone zone; bool peak; };
   Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
   for (const Trip& t : trips) {
      long long f = fare(t.zone, t.peak);
      if (canTravel(balance, f)) {
         balance -= f;
         std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                   << std::setfill('0') << f % 100 << std::setfill(' ');
      } else {
         std::cout << "Denied (low balance)";
```

```
    }
    std::cout << "  balance Rs " << balance / 100 << '\n';
  }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two styles.

Discuss:

- A procedural version with separate variables and free functions
- An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()
- How the class prevents a negative balance
- Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

- Reading values with cin and checking cin.fail() when the operator types letters instead of numbers
- Clearing the error state with cin.clear() and discarding the bad input
- Formatting money with <iomanip> (setw, setfill, fixed, setprecision)
- Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

- Normal journeys in each zone, peak and off-peak
- Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50
- Invalid cases: an unknown zone, a negative recharge, letters typed for the hour
- A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

- Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ
- Whether int overflows for the network total
- The compiler warnings you received with -Wall -Wextra and what each meant
- Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

- The types, constants and classes you recommend and why
- How a fare revision will be carried out
- The limitations of your design
- Whether the design remains correct for the whole network

Your recommendation must be based on evidence from your own experiments.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Object-Oriented Analysis	K5 – Evaluate	4
Task 2 – Tokens and Constants	K5 – Evaluate	3
Task 3 – Data Types and Compatibility	K5 – Evaluate	4
Task 4 – Implicit Conversions	K5 – Evaluate	4
Task 5 – Operators and Precedence	K5 – Evaluate	4
Task 6 – Control Structures	K5 – Evaluate	4
Task 7 – Procedural vs Object-Oriented	K5 – Evaluate	4
Task 8 – Input and Output	K5 – Evaluate	4
Task 9 – Implementation & Testing	K5 – Evaluate	4
Task 10 – Expectation vs Execution	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		42
Task 12 – Day-4 Design Challenge	K5 – Evaluate	8
TOTAL		50

Submission Requirements

- GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

ASSIGNMENT 1 - ANSWER REPORT

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Name	Monika.S
Register No.	1924260182
Institution	SIMATS Engineering
Course	Object Oriented Programming with C++ - CO1
Compiler	g++ 13.3.0 (Ubuntu 24.04)
Compile command	g++ -std=c++17 -Wall -Wextra taskN.cpp -o taskN
GitHub Repository	DSA01-1924260182-Assignment1 (paste your link here: _____)

Repository layout: Task01 ... Task12 folders, each with source, input files and output screenshots; README.md with name, register number, compiler, compile command, run steps and result of each task. Commit once per task (e.g. "Task06: switch-based fare() completed and tested").

Assumptions: (1) concession = 25% for students/senior citizens (the assignment does not give a rate); (2) peak = 08:00-09:59 and 17:00-19:59, i.e. [8,10) and [17,20); (3) all money is stored in integer paise; (4) a journey is charged by the farthest zone travelled; (5) all outputs shown in this report were produced by compiling and running the code with the command above.

Note: For Tasks 4 and 5 the "Predict the Output" step must be written by you BEFORE running; the predictions below are derived by hand from C++ rules - write them in your own hand/notes first, then compare with the real run.

TASK 1 - Understand the Problem and Identify the Objects (4 marks)

1.1 Objects, data and behaviour

Object	Data (private)	Behaviour	Justification
SmartCard	balance (long long paise)	canTravel(fare), deduct(fare), recharge(amount), balance()	Owns the money; only it may change the balance, so the "never below Rs 50" rule lives in one place.
Passenger	category (normal / student / senior), card	isConcession()	Concession depends on the person, not on the journey or the card.
Journey	from-zone, to-zone, hour, weekend flag	farthestZone(), isPeak()	A journey is one event; peak status is computed from its hour.
Zone	Inner / Middle / Outer (enum class)	-	A fixed closed set of values; an enum class prevents mixing with ints.
FareTable	base fares per zone, surcharge %, concession %, min balance	baseFare(zone), compute(journey, passenger)	All fare numbers in ONE place; a revision touches only this object.

1.2 Procedural vs object-oriented

Procedural: global variables (balance, fares) and free functions such as *fare(zone,peak)* and *deduct(balance,f)*. Any function can modify the global balance, so a bug elsewhere can make it negative (exactly the defect reported in the scenario), and fare values get copied into several functions. **Object-oriented:** data and the functions that protect it are bundled in a class; the balance is private and can only change through deduct(), which checks the rule first. The program is organised around the things in the problem (card, journey, fare table) instead of a sequence of steps.

1.3 OOP principles in this system

- **Encapsulation:** SmartCard hides balance_; only deduct()/recharge() change it.
- **Abstraction:** the gate calls card.canTravel(f) without knowing how the minimum-balance rule is implemented.
- **Inheritance:** StudentCard and SeniorCard (or Passenger types) can inherit from SmartCard / a Concession base class and add only their own discount.
- **Polymorphism:** a virtual FareRule::compute() with PeakRule, WeekendRule, PassRule - the gate calls the same function and the right rule runs.

1.4 Rules every correct journey must satisfy

- R1: balance after fare >= Rs 50 (5000 paise); otherwise the journey is denied and nothing is deducted.
- R2: balance is never negative (recharge amount must be > 0; the constructor rejects negative balance).
- R3: fare = base fare of the farthest zone, +20% only in peak hours [8,10) and [17,20) - one half-open rule so 10:00 and 20:00 are consistently OFF-peak.
- R4: all money is exact integer paise - no floating point rounding.
- R5: zone and hour must be valid (zone 0-2, hour 0-23) before a fare is computed.
- R6: fare values are defined in one place only.

Alternative Concept and Complexity Analysis

Procedural alternative: global long long balance; global fare constants; free functions fare(), canTravel(), deduct(balance&, f). **Complexity:** time per journey is O(1) in both (a switch, a few arithmetic operations, one comparison); memory per card is O(1) - 8 bytes for the balance in both (a class with one long long member is still 8 bytes). **Maintainability:** if one fare rule changes (e.g. minimum balance), the OO version changes one constant/one method; the procedural version must be searched for every function and every place the value was typed, and nothing stops other code from bypassing the check. With many cards the procedural design needs arrays of globals, whereas OO simply creates more SmartCard objects.

TASK 2 - Tokens, Keywords, Identifiers and Constants (3 marks)

2.1 The five kinds of tokens (from the program in Task 6)

Token kind	Examples in the program
Keywords	enum, class, constexpr, long, long, bool, switch, case, default, return, if, else, for, const, int
Identifiers	Zone, BASE_PAISE, fare, canTravel, balance, MIN_BALANCE_PAISE, peak
Constants (literals)	1000, 20, 5000, 100, 10000, 0, true, false
Operators	=, +, +=, -=, *, /, %, <, >=, <<, ::
Punctuators	; , ( ) { } [ ] : #

2.2 #define vs const vs constexpr; enum vs enum class

Feature	#define	const	constexpr
Type checking	None (text substitution)	Yes	Yes
Scope	Whole file after definition; ignores namespaces/blocks	Normal C++ scope	Normal C++ scope
Evaluated at	Preprocessor	Run time or compile time	Guaranteed compile time
Usable for array sizes / static_assert	Yes	Only if constant-initialised	Always
Debugging	Name not visible to debugger	Visible	Visible

constexpr is preferred for the fare table because the values are known at compile time, are type-checked, scoped, can be used in static_assert and array sizes, and cost no run-time work. **enum class** Zone { Inner, Middle, Outer } is safer than a plain enum: names are scoped (Zone::Inner), there is no implicit conversion to int, and two enums cannot clash or be compared by mistake. A plain enum would allow *Zone z = 7;*-style accidents and pollute the global namespace.

2.3 Constants used (single revision point)

```
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int  PEAK_SURCHARGE_PERCENT = 20;
constexpr int  CONCESSION_PERCENT     = 25;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

2.4 Naming rules and conventions

Identifiers may contain letters, digits and underscore, cannot start with a digit, cannot be a keyword, and are case-sensitive; names beginning with underscore + capital or containing double underscore are reserved. Convention used: constants UPPER_SNAKE_CASE (MIN_BALANCE_PAISE), types PascalCase (SmartCard, Zone), functions/variables camelCase (canTravel), private members with trailing underscore (balance_), and names that carry the unit (_PAISE, _PERCENT).

Alternative Concept and Complexity Analysis

With #define (#define BASE_INNER 1000) the preprocessor blindly replaces text: no type (an expression like BASE_INNER/3 may be int arithmetic by accident), no scope, no debugger name, and unparenthesised macros such as #define PEAK 100+20 break precedence (PEAK*2 becomes 100+40). Zones as #define 0/1/2 can be mixed with any int. **Memory:** a macro uses no storage; a constexpr scalar usually also uses none (the compiler folds it into the instruction). **Time:** identical machine code - no running-time difference. The gain of constexpr/enum class is safety, not speed.

TASK 3 - Data Types and Type Compatibility (4 marks)

3.1 sizeof and ranges on my system (g++ 13, 64-bit Linux)

Type	sizeof	Range / note
bool	1	false / true
char	1	-128..127
int	4	-2,147,483,648 .. 2,147,483,647 (INT_MIN..INT_MAX)
unsigned int	4	0 .. 4,294,967,295
long long	8	about +/- 9.22 x 10^18 (LLONG_MAX = 9223372036854775807)
float	4	~7 significant digits
double	8	~15-16 significant digits

3.2 Data-type table for the system

Value	Type	Reason
Fare, balance, surcharge amount	long long (paise)	Exact; money never fractional in paise; large range
Percent (20, 25)	int	Small whole number
Hour of day (0-23)	int	Used in comparisons; small
Zone number	enum class Zone (underlying int)	Closed set, type-safe
peak, concession, weekend	bool	Yes/no flags
Journey count per day	long long	3,000,000 per day fits int, but yearly totals do not
Day's network collection	long long (paise)	Overflows int (see 3.4)
Averages / reports only	double	Only for display, never for stored money

3.3 Why integer paise and not double rupees

Most decimal fractions (0.1, 0.2, 12.34) have no exact binary representation, so double rupees carry tiny errors that accumulate and can make equality tests such as *balance - fare == 50.0* fail. This is the "off by a paisa" bug. Integer paise are exact, and integer operators (+, -, *, /) have no rounding error (division truncates in a defined way). Evidence: *0.1 + 0.2 == 0.3* printed **false** in Task 10.

3.4 Type compatibility and overflow

In a mixed expression the compiler applies the usual arithmetic conversions: bool/char/short are promoted to int; if one operand is double the other becomes double; if int meets unsigned int, the int becomes unsigned (so -1 becomes 4,294,967,295); long long outranks int. A day's collection: 3,000,000 journeys x about Rs 12.34 = 3,702,000,000 paise, which is greater than INT_MAX (2,147,483,647), so **int is NOT enough**; long long is required.

Problem found and resolved: a 32-bit int total would hold **-592,967,296** (overflow) while the long long total is 3,702,000,000. Resolution: all money variables and accumulators are long long paise.

Alternative Concept and Complexity Analysis

double rupees: fare = 10.0 * 1.2; balance - fare >= 50.0. Time per addition is O(1) in both and space per value is 8 bytes in both. Error growth measured (adding Rs 12.34 n times): n=20,000 -> error ~0; n=500,000 -> -0.000043; n=3,000,000 -> +0.000631 rupees (see Task 10). The double error grows with n (roughly O(n) worst case) while the paise total is exactly zero-error; with float the error was 54, 11,968 and -749,764 rupees, so float is unusable for money.

TASK 4 - Control Implicit Conversions (4 marks)

4.1 Conversion points

- **Integer division:** a / b with two ints is int division (7/2 = 3); the 3 is then converted to double 3.0 on assignment.
- **Promotion:** char + int -> int, so ch + 1 prints a number (67+1 = 68), not a letter.
- **int to bool:** any non-zero value becomes true (1).
- **Signed/unsigned:** in s < u the signed int -1 is converted to unsigned 4,294,967,295, so the comparison is false. g++ -Wall -Wextra warns: *-Wsign-compare*.
- **Narrowing:** int x = 3.7; silently truncates to 3; int x{3.7}; is a compile ERROR (brace initialisation forbids narrowing).
- **static_cast** is needed when the conversion is intended: static_cast<double>(a)/b gives 3.5; static_cast<Zone>(i) converts an int to an enum class.

4.2 Predicted output (written before running)

```
3 3.5
C 68
1 true
s >= u
```

4.3 Actual output (g++ -std=c++17 -Wall -Wextra)

```
#include <iostream>
int main()
{
 int a = 7, b = 2;
 double c = a / b; double d = static_cast<double>(a) / b;
 std::cout << c << ' ' << d << '\n';
 char ch = 'A' + 2;
 std::cout << ch << ' ' << ch + 1 << '\n';
 bool flag = 5;
 std::cout << flag << ' ' << std::boolalpha << flag << '\n';
 unsigned int u = 1;
 int s = -1;
 std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Output:

```
3 3.5
C 68
1 true
s >= u
```

Compiler warning: *task4.cpp:13: warning: comparison of integer expressions of different signedness: int and unsigned int [-Wsign-compare]*.

4.4 Line-by-line explanation

Line	Output	Conversion / rule
1	3 3.5	a/b is integer division = 3, converted to double 3 (printed "3"). static_cast makes the division floating point: 3.5.
2	C 68	'A'+2 = 67 stored in char = 'C'. ch + 1: char promoted to int -> 68, so it prints as a number.
3	1 true	bool flag = 5: int to bool conversion gives true. Without boolalpha prints 1; after std::boolalpha prints "true".
4	s >= u	-1 converted to unsigned = 4294967295, which is not less than 1.

Difference prediction vs actual: none, since the prediction followed the rules above. (The only extra item is the compiler warning for line 4.)

4.5 Corrected code using braces / static_cast

```
int a{7}, b{2};
double c{static_cast<double>(a) / b};       // 3.5, intent explicit
char ch{static_cast<char>('A' + 2)};
bool flag{true};                           // int x{5} -> bool{5} would be narrowing error
unsigned int u{1}; int s{-1};
bool less = s < 0 || static_cast<unsigned>(s) < u;   // signed-safe comparison
```

Alternative Concept and Complexity Analysis

All explicit: every conversion is written with static_cast and every initialiser uses braces. Running time and memory are unchanged: the casts compile to the same instructions (or none), so O(1) time and space as before. The benefit is error detection: brace initialisation turns narrowing (double->int, int->char, large->small) into compile-time errors, and explicit casts document intent and silence/justify the -Wsign-compare and -Wconversion warnings. Implicit conversions would only show up as wrong results at run time (e.g. 3 instead of 3.5, or -1 < 1 being false).

TASK 5 - Correct Expressions and Precedence (4 marks)

5.1 Expressions for the system

```
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
fare += fare * PEAK_SURCHARGE_PERCENT / 100;           // surcharge (integer paise)
long long f = conc ? base * (100 - CONCESSION_PERCENT) / 100 : base;   // ?:
```

- **Precedence (high to low):** ( ) > ++ -- (postfix) > * / % > + - > < <= > >= > == != > && > || > ?: > = += -= ... > comma.
- **Associativity:** most binary operators left-to-right; assignment, compound assignment and ?: are right-to-left.
- **peak check:** && binds tighter than ||, so hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent, but the parentheses make the intent obvious and prevent mistakes. The half-open ranges [8,10) and [17,20) fix the 10 AM / 8 PM inconsistency.
- **surcharge:** fare * 20 / 100 evaluates left to right (* and / same precedence), keeping the multiplication first so integer division loses nothing for whole-rupee fares; fare += ... is compound assignment evaluated after the right side.
- **x++ vs ++x:** x++ yields the old value then increments; ++x increments first and yields the new value. **i = i++** modifies i twice without a sequence point (undefined behaviour before C++17, unspecified/bad style after) and should never be written.
- **Negative operands:** since C++11 / truncates toward zero and a % b has the sign of the dividend: -10/3 = -3, -10%3 = -1 (and (a/b)*b + a%b == a).

5.2 Predicted output (written before running)

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

5.3 Actual output

```
#include <iostream>
int main()
{
 int x = 5, y = 3;
 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
 int z = x++ + ++y;
 std::cout << z << ' ' << x << ' ' << y << '\n';
 std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
 int k = 0;
 for (int i = 0; i < 10; i += 3) k += i;
 std::cout << k << '\n';
}
```

Output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line	Explanation
11 16	x + y*2: * before + -> 5 + 6 = 11. (x+y)*2: parentheses first -> 8*2 = 16.
9 6 4	z = x++ + ++y: x++ gives 5 (x becomes 6), ++y gives 4 (y becomes 4); + then = : z = 9; x = 6, y = 4.
6 4	(x>y ? x : y) -> 6 > 4 so 6. x % 4 * 2: % and * same level, left to right: (6%4)*2 = 2*2 = 4.
3 1 -3 -1	10/3=3, 10%3=1, -10/3 = -3 (truncation toward zero), -10%3 = -1 (sign of dividend).
18	i = 0,3,6,9 (i += 3 while i < 10): k = 0+3+6+9 = 18.

Difference prediction vs actual: none.

Alternative Concept and Complexity Analysis

Lookup table: static constexpr bool PEAK[24] = {0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,1,1,1,0,0,0,0}; bool peak = PEAK[hour]; (hour must be validated 0-23 first). Time per check is O(1) in both; the table needs one indexed load (no comparisons) against up to 4 comparisons and 3 logical operators for the expression, so it is slightly faster and branch-free. Extra space: 24 bytes. Changing peak hours = editing table entries (easy for many irregular windows) versus editing the expression; the expression needs no array bound check and no extra memory, and is clearer for two simple windows.

TASK 6 - Control Structures (4 marks)

6.1 Choice of structures

- **if-else** for ranges and conditions (peak or not, balance check); **switch** on Zone because it is a closed set of discrete values - the compiler may build a jump table and warns (-Wswitch) if an enum value is missing; **?:** for a short two-way value choice such as concession ? discounted : full.
- **for** when the number of journeys is known (index loop); **range-based for** (for (const Trip& t : trips)) is best for processing every journey - no index errors, no copy when using const&; **while** for reading journeys until end of input (while (cin >> zone >> hour)).
- **continue** skips an invalid journey and goes to the next one; **break** leaves a switch case (and a loop on a fatal error such as end of data).
- **No case matches:** the default branch runs; without default nothing is executed and the variable keeps its old value (here we return -1, so an unknown zone cannot be charged).

6.2 Pseudocode (SIMATS Engineering style)

```
ALGORITHM ComputeFare(zone, peak)
INPUT : zone (Inner/Middle/Outer), peak (true/false)
OUTPUT: fare in paise, or -1 if zone is invalid
BEGIN
  SWITCH zone
    CASE Inner  : base <- BASE_PAISE[0]
    CASE Middle : base <- BASE_PAISE[1]
    CASE Outer  : base <- BASE_PAISE[2]
    DEFAULT     : RETURN -1
  END SWITCH
  IF peak THEN base <- base + base * PEAK_SURCHARGE_PERCENT / 100
  RETURN base
END

ALGORITHM CanTravel(balance, fare)
BEGIN
  IF fare < 0 THEN RETURN false
  RETURN (balance - fare) >= MIN_BALANCE_PAISE
END
```

6.3 Complete the Code - completed and executed

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card
long long fare(Zone z, bool peak)
{
 long long base = 0;
 switch (z) {
 case Zone::Inner:  base = BASE_PAISE[0]; break;
 case Zone::Middle: base = BASE_PAISE[1]; break;
 case Zone::Outer:  base = BASE_PAISE[2]; break;
 default: return -1; // unknown zone
 }
 if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
 return base;
}
bool canTravel(long long balance, long long f)
{
 return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
int main()
{
 long long balance = 10000; // Rs 100.00 in paise
 struct Trip { Zone zone; bool peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
 for (const Trip& t : trips) {
 long long f = fare(t.zone, t.peak);
 if (canTravel(balance, f)) {
 balance -= f;
 std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
 << std::setfill('0') << f % 100 << std::setfill(' ');
 } else {
 std::cout << "Denied (low balance)"; }
```

```
 std::cout << " balance Rs " << balance / 100 << '\n';
 }
}
```

Output:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Matches the expected output exactly: 1000 -> 10.00 (balance 90); Outer peak 3000 + 20% = 3600 -> 36.00 (balance 54); Middle 2000 would leave 54 - 20 = Rs 34 < Rs 50, so denied. Peak boundary test is in Task 9: 10:00 and 20:00 are treated as OFF-peak (3000 paise), 08:00 and 17:00 as peak (3600) - verified by the PASS lines in Task 9.

Alternative Concept and Complexity Analysis

Array lookup: constexpr long long FARE[] = {1000,2000,3000}; base = FARE[static_cast<int>(z)]; (after validating the index). Time is O(1) in both (a switch may compile to a jump table, the array is one indexed load). Space: array of 3 x 8 = 24 bytes (switch embeds the constants in code). Fourth zone: with the array just add one value (and a static_assert that the size equals the number of zones); with a switch a new case must be added in every switch over Zone - easier to forget, but -Wswitch helps. The array version scales better; the switch gives per-zone custom logic and a built-in default for invalid values.

TASK 7 - From Procedural to Object-Oriented (4 marks)

7.1 Procedural version

```
// procedural: balance is a global; any function can change it
constexpr long long MIN_BALANCE_PAISE = 5000;
long long balance = 10000;
bool canTravel(long long bal, long long f) { return bal - f >= MIN_BALANCE_PAISE; }
void deduct(long long f) { if (canTravel(balance, f)) balance -= f; }
// danger: anywhere else  balance = -300;  or  balance -= 9000;  is legal
```

7.2 Object-oriented version (SmartCard)

```
class SmartCard {
    long long balance_;                         // private: cannot be set directly
public:
    explicit SmartCard(long long b) : balance_(b >= 0 ? b : 0) {}
    long long balance() const { return balance_; }
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE_PAISE; }
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return true; }
    bool recharge(long long a) { if (a <= 0) return false; balance_ += a; return true; }
};
```

This class is the one executed in Task 9 (all 18 tests PASS). **How it prevents a negative balance:** balance_ is private, the constructor clamps a negative start value, recharge() rejects zero/negative amounts, and deduct() is the only way to reduce the balance and it first calls canTravel(), so the balance can never drop below Rs 50 after a journey (and therefore never below 0).

7.3 Comparison

Criterion	Procedural	Object-oriented
Protect balance	Impossible - global	Guaranteed by private member + checked methods
Change min balance	Search all functions using it (or one constant, but no protection)	One constant used by one method
Many cards	Parallel arrays/globals	Just create SmartCard objects
Testing	Depends on global state	Each object tested in isolation
Code size	Slightly shorter	A little more code, far safer

Alternative Concept and Complexity Analysis

Public data member: struct SmartCard { long long balance; }; Time per operation and space per card are identical (O(1), 8 bytes). But any part of the program can write card.balance = -100; or card.balance -= fare; without the check, so the rules *balance never negative* and *at least Rs 50 must remain* can be broken by any code; a bug or a careless line anywhere defeats them, and the check cannot be changed in one place. The private-member design is therefore preferred.

TASK 8 - Read and Print Data Safely (4 marks)

8.1 Safe input with cin

```
#include <iostream>
#include <limits>
#include <iomanip>
// read an int in [lo,hi]; keep asking until valid
int readInt(const char* prompt, int lo, int hi)
{
    int v;
    while (true) {
        std::cout << prompt;
        if (std::cin >> v && v >= lo && v <= hi) return v;
        if (std::cin.eof()) return lo;              // end of input: stop
        std::cout << "Invalid input, try again.\n";
        std::cin.clear();                           // clear the fail state
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // discard bad line
    }
}
void printMoney(long long paise)
{
    std::cout << "Rs " << paise / 100 << '.' << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}
int main()
{
    int zone = readInt("Zone (0=Inner,1=Middle,2=Outer): ", 0, 2);
    int hour = readInt("Hour (0-23): ", 0, 23);
    long long f = 3600;
    std::cout << std::left << std::setw(10) << "Zone" << std::right << std::setw(6) << "Hour"
              << std::setw(12) << "Fare" << '\n';
    std::cout << std::left << std::setw(10) << zone << std::right << std::setw(6) << hour << std::setw(5) << "";
    printMoney(f); std::cout << '\n';
}
```

Demonstrated recovery: input "abc" for hour sets failbit, cin >> v returns false, cin.fail() is true; cin.clear() resets the stream state and cin.ignore(max,'\n') throws away the remaining bad characters, so the next prompt reads normally. Without clear(), every later read would silently fail and loop forever. Sample session: Hour: abc -> "Invalid input, try again." -> 25 -> "Invalid input, try again." -> 9 -> accepted; report line "Zone 2, Hour 9, Fare Rs 36.00".

Formatting: setw(n) sets minimum width for the next item, setfill(c) the padding character (remember to restore it to ' '), fixed + setprecision(2) prints doubles with two decimals - but for money we avoid doubles and print paise as rupees = p/100, paise = p%100 with setw(2)/setfill('0') (so 5 paise prints 05, not 5).

Alternative Concept and Complexity Analysis

getline + stoi in try/catch: std::string line; getline(cin, line); try { int v = std::stoi(line); } catch (const std::invalid_argument&) {...} catch (const std::out_of_range&) {...}. Time per line is O(L) in both (every character is examined once); extra space is O(L) for the string buffer versus O(1) for cin >>. Recovery: with cin >> the stream is in a fail state that must be cleared and the bad input discarded; with getline the whole line is already consumed, the stream stays healthy, and errors are handled with exceptions. Caveat: stoi accepts "12abc" (returns 12) so check the pos argument; getline is more robust for line-oriented gate input.

TASK 9 - Implement and Test the Fare Logic (4 marks)

9.1 Source code (table-driven tests, SmartCard class, validation)

```
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_PERCENT = 20;
constexpr int CONCESSION_PERCENT = 25;   // assumption: students/seniors pay 75%
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }

long long fare(Zone z, bool peak, bool concession)
{
 long long f = 0;
 switch (z) {
 case Zone::Inner:  f = BASE_PAISE[0]; break;
 case Zone::Middle: f = BASE_PAISE[1]; break;
 case Zone::Outer:  f = BASE_PAISE[2]; break;
 default: return -1;
 }
 if (peak) f = f * (100 + PEAK_PERCENT) / 100;
 if (concession) f = f * (100 - CONCESSION_PERCENT) / 100;
 return f;
}

class SmartCard {
 long long balance_;
public:
 explicit SmartCard(long long b) : balance_(b >= 0 ? b : 0) {}
 long long balance() const { return balance_; }
 bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE_PAISE; }
 bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return true; }
 bool recharge(long long a) { if (a <= 0) return false; balance_ += a; return true; }
};

// returns fare paise, -1 invalid input, -2 low balance
long long journey(SmartCard& c, const std::string& zoneStr, const std::string& hourStr, bool conc)
{
 int zi, h;
 try { zi = std::stoi(zoneStr); h = std::stoi(hourStr); }
 catch (...) { return -1; }
 if (zi < 0 || zi > 2 || h < 0 || h > 23) return -1;
 long long f = fare(static_cast<Zone>(zi), isPeak(h), conc);
 return c.deduct(f) ? f : -2;
}

struct Test { const char* name; const char* zone; const char* hour; bool conc; long long startBal; long long expected; };

int main()
{
 std::vector<Test> tests = {
  {"Inner off-peak 12:00",      "0", "12", false, 10000, 1000},
  {"Middle off-peak 12:00",     "1", "12", false, 10000, 2000},
  {"Outer off-peak 12:00",      "2", "12", false, 10000, 3000},
  {"Inner peak 09:00",          "0", "9",  false, 10000, 1200},
  {"Middle peak 08:00 (start)", "1", "8",  false, 10000, 2400},
  {"Outer peak 17:00 (start)",  "2", "17", false, 10000, 3600},
  {"Outer 10:00 (end, off)",    "2", "10", false, 10000, 3000},
  {"Outer 20:00 (end, off)",    "2", "20", false, 10000, 3000},
  {"Outer 07:00 off-peak",      "2", "7",  false, 10000, 3000},
  {"Student Middle off-peak",   "1", "12", true,  10000, 1500},
  {"Senior Outer peak",         "2", "18", true,  10000, 2700},
  {"Leaves exactly Rs 50",      "0", "12", false, 6000,  1000},
  {"Leaves Rs 49.99 (denied)",  "0", "12", false, 5999,  -2},
  {"Unknown zone 5",            "5", "12", false, 10000, -1},
  {"Letters for hour",          "0", "ab", false, 10000, -1},
  {"Hour 25",                   "0", "25", false, 10000, -1},
 };
 int pass = 0;
 std::cout << std::left << std::setw(28) << "Case" << std::right << std::setw(10) << "Expected"
           << std::setw(10) << "Actual" << "  Result\n";
 for (const Test& t : tests) {
  SmartCard c(t.startBal);
  long long a = journey(c, t.zone, t.hour, t.conc);
  bool ok = (a == t.expected) && c.balance() >= 0;
```

```
  pass += ok;
  std::cout << std::left << std::setw(28) << t.name << std::right << std::setw(10) << t.expected
            << std::setw(10) << a << "  " << (ok ? "PASS" : "FAIL") << '\n';
 }
 SmartCard neg(10000);
 bool r = neg.recharge(-500);
 std::cout << "Negative recharge rejected: " << (!r && neg.balance() == 10000 ? "PASS" : "FAIL") << '\n';
 pass += (!r && neg.balance() == 10000);

 // five journeys, total calculated by hand = 1000+2400+3000+1200+1500 = 9100 paise
 SmartCard five(100000);
 long long total = 0;
 total += journey(five, "0", "12", false);
 total += journey(five, "1", "8",  false);
 total += journey(five, "2", "11", false);
 total += journey(five, "0", "17", false);
 total += journey(five, "1", "14", true);
 std::cout << "Five-journey total: Rs " << total / 100 << '.' << std::setw(2) << std::setfill('0')
           << total % 100 << std::setfill(' ') << " (hand: Rs 91.00) "
           << (total == 9100 ? "PASS" : "FAIL") << '\n';
 pass += (total == 9100);
 std::cout << pass << " / " << tests.size() + 2 << " passed\n";
}
```

9.2 Test results (expected vs actual)

Output:

```
Case                          Expected    Actual  Result
Inner off-peak 12:00              1000      1000  PASS
Middle off-peak 12:00             2000      2000  PASS
Outer off-peak 12:00              3000      3000  PASS
Inner peak 09:00                  1200      1200  PASS
Middle peak 08:00 (start)         2400      2400  PASS
Outer peak 17:00 (start)          3600      3600  PASS
Outer 10:00 (end, off)            3000      3000  PASS
Outer 20:00 (end, off)            3000      3000  PASS
Outer 07:00 off-peak              3000      3000  PASS
Student Middle off-peak           1500      1500  PASS
Senior Outer peak                 2700      2700  PASS
Leaves exactly Rs 50              1000      1000  PASS
Leaves Rs 49.99 (denied)            -2        -2  PASS
Unknown zone 5                      -1        -1  PASS
Letters for hour                    -1        -1  PASS
Hour 25                             -1        -1  PASS
Negative recharge rejected: PASS
Five-journey total: Rs 91.00 (hand: Rs 91.00) PASS
18 / 18 passed
```

Coverage: every zone peak/off-peak; boundaries 08:00 and 17:00 (peak), 10:00 and 20:00 (off-peak); a balance that leaves exactly Rs 50 (allowed) and Rs 49.99 (denied); invalid zone, hour typed as letters, hour 25; negative recharge rejected; five-journey total calculated by hand: 1000 + 2400 + 3000 + 1200 + 1500 = 9100 paise = Rs 91.00 and matched by the program.

Alternative Concept and Complexity Analysis

Table of test cases in a loop (used above): time for t tests is O(t) (each test O(1)); the table needs O(t) space (about 40 bytes per row). Adding a new boundary case = adding one line to the table; with separate hand-written checks you would copy a whole block of code (and risk copy-paste mistakes), and totals/pass-counts have to be maintained manually. Table-driven tests are shorter, uniform and easy to extend or load from a file.

TASK 10 - Compare Expectations with Execution (4 marks)

```
#include <iostream>
#include <iomanip>
#include <climits>
int main()
{
 const long long n[] = {20000, 500000, 3000000};
 std::cout << "INT_MAX = " << INT_MAX << ", LLONG_MAX = " << LLONG_MAX << '\n';
 std::cout << "0.1+0.2 == 0.3 ? " << std::boolalpha << (0.1 + 0.2 == 0.3) << '\n';
 for (long long cnt : n) {
  double d = 0; long long p = 0; float f = 0;
  for (long long i = 0; i < cnt; ++i) { d += 12.34; f += 12.34f; p += 1234; }
  double exact = p / 100.0;
  int wrapped = static_cast<int>(static_cast<unsigned int>(p)); // what a 32-bit int would hold
  std::cout << std::fixed << std::setprecision(6) << cnt
   << " journeys | paise total=" << p << " (Rs " << exact << ")"
   << " | double=" << d << " diff=" << (d - exact)
   << " | float diff=" << (f - exact)
   << " | 32-bit int would hold " << wrapped << (p > INT_MAX ? " OVERFLOW" : " ok") << '\n';
 }
}
```

Output:

```
INT_MAX = 2147483647, LLONG_MAX = 9223372036854775807
0.1+0.2 == 0.3 ? false
20000 journeys | paise total=24680000 (Rs 246800.000000) | double=246800.000000 diff=-0.000000 | float diff=54.312500 | 3
500000 journeys | paise total=617000000 (Rs 6170000.000000) | double=6169999.999957 diff=-0.000043 | float diff=11968.000
3000000 journeys | paise total=3702000000 (Rs 37020000.000000) | double=37020000.000631 diff=0.000631 | float diff=-74976
```

Journeys	Exact total (paise accumulator)	double total (Rs)	double error (Rs)	float error (Rs)	32-bit int
20,000	Rs 246,800.00	246800.000000	about 0	54.31	ok
500,000	Rs 6,170,000.00	6169999.999957	-0.000043	11,968	ok (617,000,000)
3,000,000	Rs 37,020,000.00	37020000.000631	+0.000631	-749,764	OVERFLOW -> -592,967,296

Discussion

- **Do double and paise totals agree?** Practically yes for double (error below one hundredth of a paisa) but not exactly; the error grows with the number of additions (0 -> -0.000043 -> +0.000631 rupees) and can flip an equality check. The paise total is exact.
- **Does int overflow?** Not at 20,000 or 500,000 journeys (24.68 million and 617 million paise), but YES at 3,000,000 (3.702 billion > 2.147 billion); signed overflow is undefined behaviour, in practice it wrapped to a negative total.
- **Compiler warnings (-Wall -Wextra):** the only warning was -Wsign-compare in Task 4 (comparing int with unsigned int: the signed value is converted to unsigned). It warns that the result may be surprising. Other things -Wall -Wextra would report in careless code: -Wunused-variable, -Wconversion/-Wnarrowing for implicit narrowing, -Wswitch for missing enum cases, -Wreturn-type when a non-void function has a path without return (e.g. the TODO functions left empty).
- **Reasons for differences:** binary floating point cannot represent 12.34 exactly; each addition rounds; float has only ~7 digits so it is far worse; int has only 32 bits.

Alternative Concept and Complexity Analysis

At 3,000,000 journeys: int fails by OVERFLOW (value outside range: wrong sign/size, undefined behaviour); **long long** is exact (maximum 9.22 x 10^18 paise, i.e. enough for billions of years of collection); **double** does not overflow but loses precision (error +0.000631 Rs here, and exactness lost once totals exceed 2^53 paise-equivalent). Time per addition is O(1) for all three (integer add is a single instruction; double add is also one, with slightly higher latency) and space per variable is 4, 8 and 8 bytes respectively. So long long gives exact results at no extra cost.

TASK 11 - Recommend and Defend the Final Design (4 marks)

11.1 Recommendation

- **Types:** long long for all money (paise) and totals; int for percentages and hour; bool flags; enum class Zone. No double/float for money.
- **Constants:** all in one header as constexpr (fare table, PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT, MIN_BALANCE_PAISE), with a static_assert that the fare table has one entry per zone.
- **Classes:** SmartCard (private balance; canTravel/deduct/recharge), a FareTable/fare() free function for computation, isPeak() with half-open ranges, validation function for input; table-driven test suite.
- **Fare revision procedure:** edit the constexpr value(s) in the single constants header, recompile with -Wall -Wextra, run the test table (expected values updated by hand), commit with a message, deploy to the gates.

11.2 Evidence from my experiments

- Task 9: 18/18 tests PASS including all boundary and invalid cases; balance never went below Rs 50 or below 0.
- Task 10: int overflowed at 3,000,000 journeys (-592,967,296) while long long gave 3,702,000,000 paise; double accumulated error up to 0.000631 Rs; float error up to Rs 749,764.
- Task 6: expected output reproduced exactly with integer paise.

11.3 Limitations

- Fares are compiled in: every revision needs recompile and redeployment.
- Integer division truncates (e.g. 25% concession on an odd paise value rounds down by < 1 paisa); the rounding policy must be agreed with the finance department.
- Input validation is by range; no time-zone, midnight-crossing or multi-line network logic is included; a single process is assumed (no concurrent updates of one card).
- The OO design does not yet model passes or weekends (addressed in Task 12).

11.4 Correct for the whole network?

Yes: per journey the work is O(1) and the memory per card is O(1), so 3,000,000 journeys a day is O(n) total with only a few operations each; all totals use long long (max ~9.22 x 10^18 paise); and the exact integer arithmetic means the result does not depend on the number of journeys. One condition: per-gate and per-line totals must also be long long.

Alternative Concept and Complexity Analysis

Fare table in a configuration file: read z values at start-up (for z = 3 or 4 zones: one small file read, O(z) time, microseconds, and O(z) memory - 8 bytes per zone, e.g. 32 bytes). Fare revision then needs only a file edit and gate restart, no recompilation or redeployment of binaries. Costs: a file can be missing/corrupt/non-numeric, so values must be validated (non-negative, correct count, sane range) and a safe default or refusal to start is needed; the values lose compile-time checking (no static_assert, not constexpr) and become ordinary run-time data. After start-up the time per journey is unchanged, O(1). For a rarely-changing 3-4 value table compile-time is simpler and safer; for frequent revisions across thousands of gates the config file is more practical.

TASK 12 - Day-4 Design Challenge: Re-evaluate and Adapt (8 marks)

Changed requirement: fourth zone Airport (Rs 60); 10% weekend discount that does not combine with the peak surcharge; monthly pass with unlimited journeys in two chosen zones.

A. Impact Analysis (2 marks)

Element	Affected?	What must change
Constants	Yes	BASE_PAISE gets a 4th entry (6000); new WEEKEND_DISCOUNT_PERCENT = 10; (MIN_BALANCE_PAISE, PEAK_SURCHARGE_PERCENT unchanged)
enum class Zone	Yes	Add Airport (and a COUNT sentinel)
switch in fare()	Yes	New case Zone::Airport; every other switch over Zone (reports, validation) must also be updated
Peak expression isPeak()	No	Unchanged
Surcharge expression	Yes	Must be skipped on weekends (rule: weekend OR peak, weekend wins)
Input validation	Yes	Zone range 0..2 becomes 0..3
SmartCard	No / small	Balance rules unchanged. Pass is a different concept; it should be a new class, not stored as more flags in SmartCard
Tests	Yes	New cases for Airport, weekend, weekend+peak hour, pass covered / not covered

B. Evaluation of the Original Design (2 marks)

- **Fourth zone:** the switch absorbs it with one extra case (small change) but the zone count is hard-coded in several places (range checks 0..2), so an update can be missed. *Worked example:* Inner->Airport off-peak weekday = Rs 60 -> with the old code returns -1 (default), i.e. the journey is wrongly refused.
- **Weekend discount:** the old fare() has only a bool peak. A second bool weekend adds nested ifs; if both are true the original code would apply +20% and -10% in some order giving 1.2 x 0.9 = 1.08 (Rs 21.60 for an Rs 20 fare) instead of the required Rs 18.00 (weekend wins, no combination). The logic must be restructured.
- **Monthly pass:** SmartCard has only a balance and cannot say "this passenger may travel free in zones X and Y"; adding pass fields to it breaks single responsibility and makes canTravel() ambiguous (a free journey needs no minimum balance? - a policy question).
- **Verdict:** the core ideas (paise, constexpr, enum class, private balance) are sound and should be RETAINED; the fare computation should be MODIFIED (table + separate rule functions); the pass is a NEW class. A full redesign is not needed.

C. Adaptation (2 marks) - implemented and tested

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer, Airport, COUNT };
constexpr long long FARE_PAISE[] = {1000, 2000, 3000, 6000};   // one place to revise
static_assert(sizeof(FARE_PAISE) / sizeof(FARE_PAISE[0]) == static_cast<int>(Zone::COUNT), "fare table must cover every z
constexpr int PEAK_PERCENT = 20, WEEKEND_DISCOUNT_PERCENT = 10, CONCESSION_PERCENT = 25;
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }

// Separate rule: weekend discount and peak surcharge never combine (weekend wins)
long long timeAdjust(long long base, bool peak, bool weekend)
{
 if (weekend) return base * (100 - WEEKEND_DISCOUNT_PERCENT) / 100;
 if (peak)    return base * (100 + PEAK_PERCENT) / 100;
 return base;
}

class Pass {                        // monthly pass: unlimited travel inside two chosen zones
 unsigned mask_ = 0;
public:
 Pass() = default;
 Pass(Zone a, Zone b) : mask_((1u << static_cast<int>(a)) | (1u << static_cast<int>(b))) {}
 bool covers(Zone from, Zone to) const {
  return (mask_ >> static_cast<int>(from) & 1u) && (mask_ >> static_cast<int>(to) & 1u);
```

```
 }
};

long long fare(Zone from, Zone to, int hour, bool weekend, bool concession, const Pass& pass)
{
 if (pass.covers(from, to)) return 0;
 Zone far = static_cast<int>(from) > static_cast<int>(to) ? from : to;  // farthest zone
 long long f = timeAdjust(FARE_PAISE[static_cast<int>(far)], isPeak(hour), weekend);
 if (concession) f = f * (100 - CONCESSION_PERCENT) / 100;
 return f;
}

class SmartCard {
 long long bal_;
public:
 explicit SmartCard(long long b) : bal_(b >= 0 ? b : 0) {}
 long long balance() const { return bal_; }
 bool canTravel(long long f) const { return f >= 0 && bal_ - f >= MIN_BALANCE_PAISE; }
 bool deduct(long long f) { if (!canTravel(f)) return false; bal_ -= f; return true; }
};

int main()
{
 Pass none, pass(Zone::Inner, Zone::Middle);
 struct T { const char* n; Zone a, b; int h; bool we, co; const Pass* p; long long exp; };
 T t[] = {
  {"Airport off-peak weekday",   Zone::Inner,  Zone::Airport, 12, false, false, &none, 6000},
  {"Airport peak weekday",       Zone::Inner,  Zone::Airport, 9,  false, false, &none, 7200},
  {"Airport weekend",            Zone::Inner,  Zone::Airport, 12, true,  false, &none, 5400},
  {"Outer weekend at peak hour", Zone::Inner,  Zone::Outer,   8,  true,  false, &none, 2700},
  {"Middle weekend student",     Zone::Inner,  Zone::Middle,  12, true,  true,  &none, 1350},
  {"Pass: Inner->Middle",        Zone::Inner,  Zone::Middle,  9,  false, false, &pass, 0},
  {"Pass: Middle->Outer (not covered)", Zone::Middle, Zone::Outer, 12, false, false, &pass, 3000},
 };
 int ok = 0;
 for (const T& x : t) {
  long long f = fare(x.a, x.b, x.h, x.we, x.co, *x.p);
  bool good = f == x.exp; ok += good;
  std::cout << std::left << std::setw(36) << x.n << std::right << " expected " << std::setw(5) << x.exp
            << " actual " << std::setw(5) << f << (good ? "  PASS" : "  FAIL") << '\n';
 }
 SmartCard c(6000);
 std::cout << "Airport peak on Rs 60 card allowed? " << std::boolalpha << c.canTravel(7200) << '\n';
 std::cout << ok << " / 7 passed\n";
}
```

Output:

```
Airport off-peak weekday             expected  6000 actual  6000  PASS
Airport peak weekday                 expected  7200 actual  7200  PASS
Airport weekend                      expected  5400 actual  5400  PASS
Outer weekend at peak hour           expected  2700 actual  2700  PASS
Middle weekend student               expected  1350 actual  1350  PASS
Pass: Inner->Middle                  expected     0 actual     0  PASS
Pass: Middle->Outer (not covered)    expected  3000 actual  3000  PASS
Airport peak on Rs 60 card allowed? false
7 / 7 passed
```

Design changes: (1) fare table FARE_PAISE[] indexed by Zone, with static_assert that its size equals Zone::COUNT, so a missing fare is a compile error; (2) timeAdjust() holds the rule "weekend discount OR peak surcharge, never both"; (3) Pass class stores the two chosen zones as a bit mask and covers(from,to) returns true only if both zones are in the pass; (4) SmartCard unchanged. Worked examples verified by the tests: Airport weekday off-peak 6000; Airport peak 7200; Airport weekend 5400 (not 7200 x 0.9); Outer at 08:00 on a weekend 2700 (surcharge not applied); student weekend Middle 1350; pass-covered journey 0; Middle->Outer with an Inner/Middle pass = 3000 (not covered). A Rs 60 card cannot take the Rs 72 Airport peak fare (min balance rule still works).

D. Evidence-Based Defence (2 marks)

- **Decision: Retain the foundations, modify fare logic, add Pass.** All 7 new tests pass and the original 18 Task 9 tests are unaffected by the new rules (same constants, same SmartCard).
- **Trade-offs:** table lookup is O(1), 32 bytes, and scales to a fifth zone by adding one value; the old switch costs a case per zone and per switch. A full redesign (e.g. virtual FareRule classes for peak/weekend/pass) would be more extensible for future rules but adds classes, heap/virtual-call overhead and complexity not justified by three small rule changes - it is the recommended next step only if rules keep growing.

- **Risks:** pass-covered journeys are free in this model (a policy decision about the Rs 50 minimum balance should be confirmed); public holidays and monthly pass expiry dates are not modelled; rounding of percentage discounts truncates in the passenger's disfavour by less than 1 paisa.
- **Complexity:** per journey still O(1) time and O(1) memory per card (+4 bytes for the pass mask); the network total remains long long and safe for 3,000,000 journeys a day.

Appendix - GitHub Submission Checklist

- Repository: DSA01-1924260182-Assignment1 (public, or private with the faculty added as collaborator).
- Folders: Task01 ... Task12 (Task04/05 predictions written BEFORE running, with screenshots; Task06 task06.cpp; Task09 task09.cpp; Task10 task10.cpp; Task12 task12.cpp).
- README.md: name, register number, g++ 13, command *g++ -std=c++17 -Wall -Wextra taskN.cpp -o taskN*, run steps, result of each task.
- Commit at least once per task with meaningful messages (history is checked). Take your own screenshots of the program output.